



Universidade Federal de Uberlândia
Faculdade de Computação



Análise Sintática (Gerador Automático Yacc)

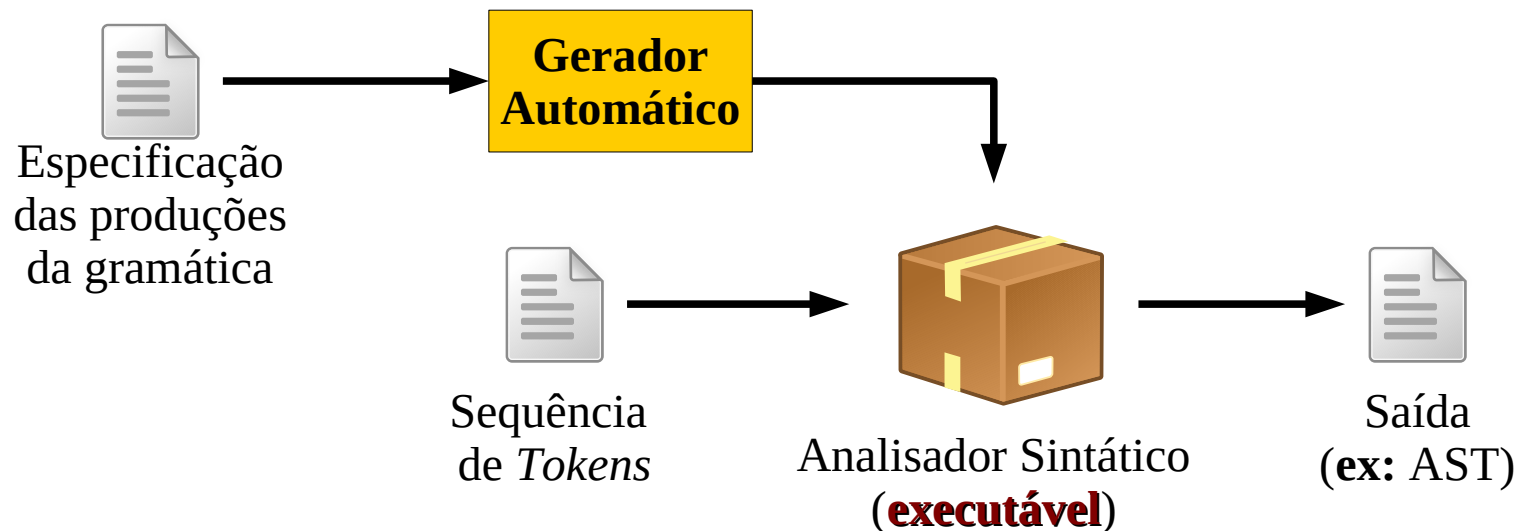
Curso de Bacharelado em Ciência da Computação
GBC071 - Construção de Compiladores
Prof. Luiz Gustavo Almeida Martins

Geradores Automático

- **Gerador de analisadores sintáticos** [Alexandre, 2014]:

*“Programa que recebe como entrada a **especificação da estrutura sintática** de uma linguagem e produz como saída um programa que faz a **análise sintática** dessa linguagem”*

- Por que usar um gerador de analisadores?
 - **Menos trabalho**
 - Garante a geração de um analisador sintático **correto**



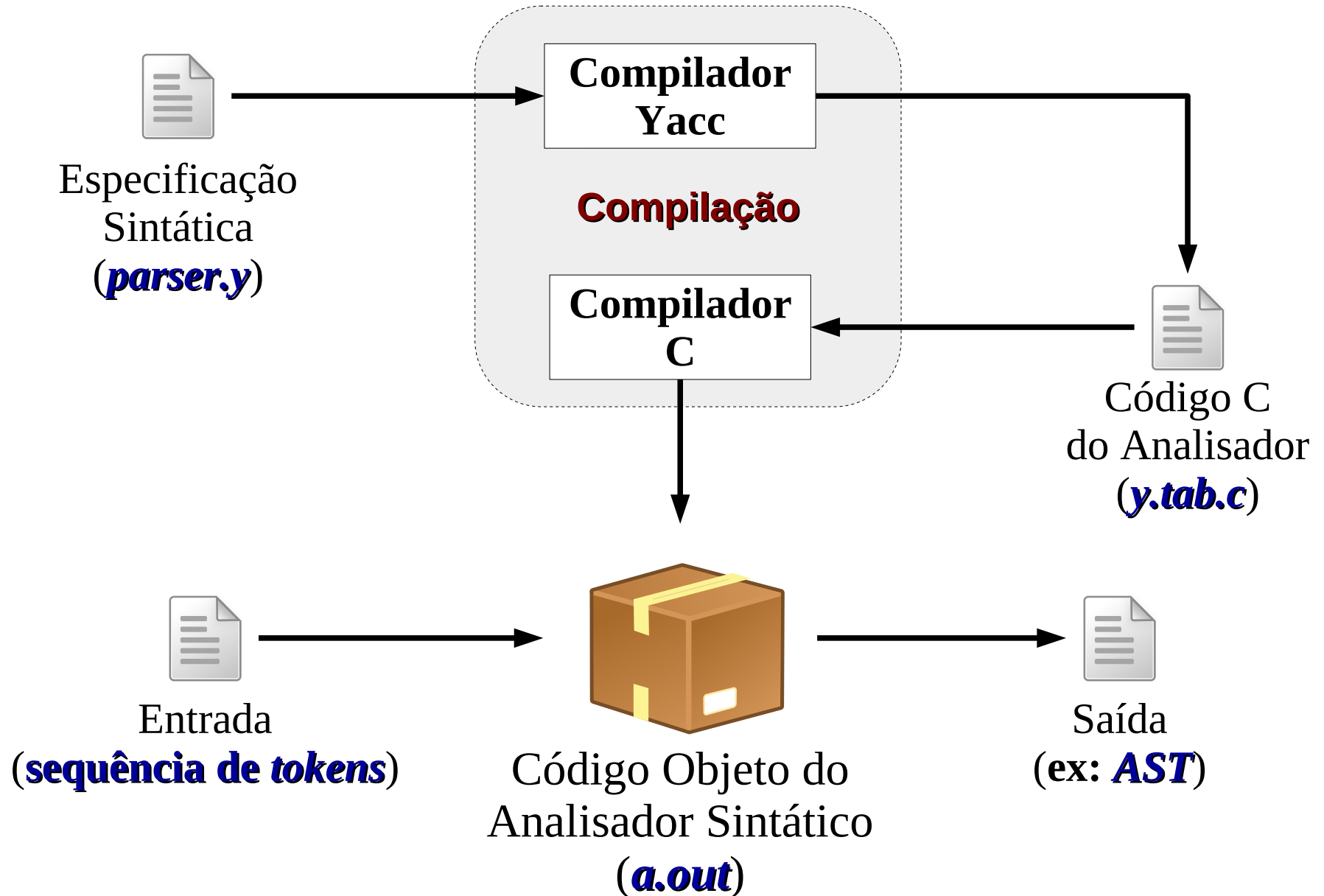
Gerador Automático Yacc

- **Yacc** (*Yet Another Compiler-Compiler*)
 - 1ª versão criada no início da década de 1970 por S. C. Johnson
 - Foi originalmente projetado para ser complementado pelo Lex
 - Ferramenta mais comum para a geração de analisadores sintáticos
 - Usado para auxiliar na implementação de compiladores de produção
 - Sua versão do projeto GNU é o **bison**
 - **javacc** é uma versão para a linguagem java
- Funciona de **forma similar e complementar** aos geradores de analisadores léxicos (**ex:** lex)
 - **yacc** gera a função **yyparse()** que analisa os itens léxicos e decide se eles formam ou não uma sentença válida
 - Deve ser invocada pela aplicação (função *main*)
 - Retorna 0, se a entrada for reconhecida, ou 1, se houver erros sintáticos
 - **lex** gera a função **yylex()** que varre o código fonte e retorna o próximo *token*
 - Rotina invocada pelo *yyparse()*

Gerador Automático Yacc

- **Especificação do analisador sintático** envolve a descrição das **regras de tradução**
 - **Produções da gramática** da linguagem **e suas ações semânticas**
 - Especificação Yacc do tradutor é descrito em um **arquivo com extensão .y**
- **Compilador Yacc** gera programa **y.tab.c** que é uma representação do **analisador sintático** criado a partir das regras de tradução
 - Gera a **versão compactada da tabela de análise LALR** e a rotina de reconhecimento (chamada **yyparse**)
 - **Sintaxe:** *yacc -d parser.y* OU *bison -d parser.y*
 - Opção **-d** gera **arquivo cabeçalho (y.tab.h)** com as principais definições, o qual é usado para a **integração com outros módulos** (ex: analisador léxico gerado pelo *flex*)
 - Opção **-v** produz arquivo **y.output** com a **descrição textual da tabela LALR(1)** gerada
- **Compilador C** usado para gerar o **código objeto a.out**
 - Arquivo **y.tab.c** deve ser compilado junto com a **biblioteca ly** que contém o programa de análise LR
 - **Ex:** *gcc -o y.tab.c -ly* (**obs:** nome **ly** é dependente do sistema)

Gerador Automático Yacc



Estrutura dos Programas Yacc

- Formato é similar ao usado no lex (**3 seções**):

Declarações

%%

Regras de tradução

%%

Código

Estrutura dos Programas Yacc

- Formato é similar ao usado no lex (**3 seções**):

Declarações

%%

Regras de tradução

%%

Código

Seções de declarações e código **são opcionais**

Seção de Declarações

- Subdividida em **2 partes opcionais**:
 - Declarações comuns da linguagem C
 - Declarações de suporte da gramática
- **Declarações da linguagem C**:
 - Conteúdo delimitado por **%{** e **%}**
 - Envolve a declaração de **diretivas** (ex: *#include* e *#define*), **variáveis globais** e quaisquer outros temporários usados nas demais seções
 - Copia o conteúdo diretamente para o arquivo **y.tab.c**

Seção de Declarações

- **Declarações de suporte** para a gramática:
 - **Definição de *tokens*** (Ex: *%token ID*)
 - *Tokens* representados por um único caractere não precisam ser declarados
 - Yacc define automaticamente um **código numérico** para cada *token*
 - Podemos atribuir um valor explícito (Ex: *%token NUM 681*) que **não deve se sobrepor aos códigos ASCII** para evitar problemas com *tokens* de um caractere
 - **Definição da associatividade** dos operadores:
 - Esquerda (*%left*), direita (*%right*) e sem associatividade (*%nonassoc*)
 - Ex: *%left SOMA SUB*
 - **Definição da precedência** dos operadores:
 - Definida pela ordem (linha) da declaração das associatividades
 - Operadores são apresentados **da menor para a maior precedência**
 - Operadores na mesma linha possuem a mesma precedência

Seção de Regras de Tradução

- Especifica as **produções** da gramática e suas **ações semânticas**
 - Parte principal da especificação Yacc (**obrigatória**)
 - Adota notação baseada em **BNF (Backus-Naur Form)**

- **Sintaxe:**

$$\begin{array}{lll} \text{cabeça} : & \text{corpo}_1 & \{ \text{ação semântica}_1 \} \\ & | & \text{corpo}_2 \quad \{ \text{ação semântica}_2 \} \\ & \dots & \\ & | & \text{corpo}_N \quad \{ \text{ação semântica}_N \} \\ & ; & \end{array}$$

Seção de Regras de Tradução

- Cada par **<cabeça, corpo>** define uma **produção**
 - **Símbolo inicial** pode ser definido de 2 formas:
 - **Seção declarações**: *%start SIMBOLO* (forma explícita)
 - **Seção regras de tradução**: cabeça da 1ª produção especificada
 - **Símbolos não-terminais**: cadeias de letras e dígitos sem aspas não declaradas como *tokens*
 - **Símbolos terminais**: único caractere entre apóstrofos (**ex**: 'a') ou identificador do *token* definido na seção declarações
 - **Corpo vazio** indica uma **produção-ε**
 - **Ex**:

```
expr : expr SOMA expr { printf("%g + %g\n", $1, $3); }  
      | /* vazio (expr → ε) */  
      ;
```

Seção de Regras de Tradução

- **Ação semântica:** fragmento de código C que determina o que fazer quando a produção é usada
 - **Geração da AST** envolve as operações de criação do nó raiz e construção da subárvore correspondente à produção
 - **Tradução dirigida por sintaxe** envolve operações sobre os valores dos atributos associados aos símbolos na produção
 - **$$$$** : valor do atributo do símbolo da **cabeça da produção**
 - **$$N$** : valor do atributo do **N -ésimo símbolo do corpo** da produção
 - Quando **ação é omitida**, adota-se a cópia do valor do atributo do 1º símbolo do corpo como **ação padrão** (**$\{ \$\$ = \$1; \}$**)
 - Pode chamar **funções auxiliares** definidas na seção de código

Seção de Código

- Especifica funções de suporte para o analisador sintático
 - Contém a função principal ***main()***, analisador léxico e outras rotinas especificadas pelo usuário
 - Todas as funções escritas em código C
 - Função ***main()*** deve chamar a função ***yyparse()***
- Funções também podem ser especificadas em **arquivos separados**
 - Conceito de **Tipo Abstrato de Dados (TAD)**
 - **Compilação independente** dos módulos
 - Exige ligação dos códigos-objeto para a **integração dos módulos**
 - Geralmente envolve o uso de arquivos cabeçalho

Exemplo de Programa Yacc

- Considere o código que implementa uma calculadora:

```
%{ #include <ctype.h>
```

```
#include <stdio.h>
```

```
int yylex(); void yyerror(char *); %}
```

```
%token DIGITO
```

```
%%
```

```
line : expr '\n'      { printf("%d\n", $1); return 0; }
```

```
;
```

```
expr : expr '+' term { $$ = $1 + $3; }
```

```
| term
```

```
;
```

```
term : term '*' fat   { $$ = $1 * $3; }
```

```
| fat
```

```
;
```

```
fat  : '(' expr ')'    { $$ = $2; }
```

```
| DIGITO
```

```
;
```

```
%%
```

```
yylex() {
```

```
    setbuf(stdin, NULL);
```

```
    int C = getchar();
```

```
    if (isdigit(c)) {
```

```
        yylval = C - '0'; // char -> int
```

```
        return DIGITO;    }
```

```
    return C;    }
```

```
void yyerror(char * s) {
```

```
    fprintf (stderr, "%s\n", s);    }
```

```
int main(void) {
```

```
    return yyparse();    }
```

Biblioteca necessária no arquivo c
para o uso da função **isdigit()**

Exemplo de Programa Yacc

- Considere o código que implementa uma calculadora:

```
%{ #include <ctype.h>
```

```
#include <stdio.h>
```

```
int yylex(); void yyerror(char *); %}
```

```
%token DIGITO
```

```
%%
```

```
line : expr '\n'      { printf("%d\n", $1); return 0; }
```

```
;
```

```
expr : expr '+' term { $$ = $1 + $3; }
```

```
| term
```

```
;
```

```
term : term '*' fat   { $$ = $1 * $3; }
```

```
| fat
```

```
;
```

```
fat  : '(' expr ')'    { $$ = $2; }
```

```
| DIGITO
```

```
;
```

```
%%
```

```
yylex() {
```

```
    setbuf(stdin, NULL);
```

```
    int C = getchar();
```

```
    if (isdigit(c)) {
```

```
        yylval = C - '0'; // char -> int
```

```
        return DIGITO;    }
```

```
    return C;    }
```

```
void yyerror(char * s) {
```

```
    fprintf (stderr, "%s\n", s);    }
```

```
int main(void) {
```

```
    return yyparse();    }
```

Único *token* associado
a um identificador

Exemplo de Programa Yacc

- Considere o código que implementa uma calculadora:

```
%{ #include <ctype.h>
```

```
#include <stdio.h>
```

```
int yylex(); void yyerror(char *); %}
```

```
%token DIGITO
```

```
%%
```

```
line : expr '\n' { printf("%d\n", $1); return 0; }
```

```
;
```

```
expr : expr '+' term { $$ = $1 + $3; }
```

```
| term
```

```
;
```

```
term : term '*' fat { $$ = $1 * $3; }
```

```
| fat
```

```
;
```

```
fat : '(' expr ')' { $$ = $2; }
```

```
| DIGITO
```

```
;
```

```
%%
```

```
yylex() {
```

```
    setbuf(stdin, NULL);
```

```
    int C = getchar();
```

```
    if (isdigit(c)) {
```

```
        yylval = C - '0'; // char -> int
```

```
        return DIGITO;    }
```

```
    return C;    }
```

```
void yyerror(char * s) {
```

```
    fprintf (stderr, "%s\n", s);    }
```

```
int main(void) {
```

```
    return yyparse();    }
```

*Tokens de um único caractere
definidos na própria produção*

Exemplo de Programa Yacc

- Considere o código que implementa uma calculadora:

```
%{ #include <ctype.h>
```

```
#include <stdio.h>
```

```
int yylex(); void yyerror(char *); %}
```

```
%token DIGITO
```

```
%%
```

```
line : expr '\n'      { printf("%d\n", $1); return 0; }
```

```
;
```

```
expr : expr '+' term { $$ = $1 + $3; }
```

```
| term
```

```
;
```

```
term : term '*' fat   { $$ = $1 * $3; }
```

```
| fat
```

```
;
```

```
fat  : '(' expr ')'    { $$ = $2; }
```

```
| DIGITO
```

```
;
```

```
%%
```

```
yylex() {
```

```
    setbuf(stdin, NULL);
```

```
    int C = getchar();
```

```
    if (isdigit(c)) {
```

```
        yylval = C - '0'; // char -> int
```

```
        return DIGITO;    }
```

```
    return C;    }
```

```
void yyerror(char * s) {
```

```
    fprintf (stderr, "%s\n", s);    }
```

```
int main(void) {
```

```
    return yyparse();    }
```

Produções adotam a ação
semântica padrão: { \$\$ = \$1; }

Exemplo de Programa Yacc

- Considere o código que implementa uma calculadora:

```
%{ #include <ctype.h>
```

```
#include <stdio.h>
```

```
int yylex(); void yyerror(char *); %}
```

```
%token DIGITO
```

```
%%
```

```
line : expr '\n'      { printf("%d\n", $1); return 0; }
```

```
;
```

```
expr : expr '+' term { $$ = $1 + $3; }
```

```
| term
```

```
;
```

```
term : term '*' fat   { $$ = $1 * $3; }
```

```
| fat
```

```
;
```

```
fat  : '(' expr ')'    { $$ = $2; }
```

```
| DIGITO
```

```
;
```

```
%%
```

```
yylex() {
```

```
    setbuf(stdin, NULL);
```

```
    int C = getchar();
```

```
    if (isdigit(c)) {
```

```
        yylval = C - '0'; // char -> int
```

```
        return DIGITO; }
```

```
    return C; }
```

```
void yyerror(char * s) {
```

```
    fprintf (stderr, "%s\n", s); }
```

```
int main(void) {
```

```
    return yyparse(); }
```

Analizador léxico implementado
manualmente no próprio arquivo

Variável global **yylval** usada para
retornar o valor do dígito

Exemplo de Programa Yacc

- **Outra versão (+ completa):**

// Seção de Declarações

```
%{  
#include <ctype.h>  
#include <stdio.h>  
  
#define YYSTYPE double // Define tipo da pilha  
  
int yylex();  
void yyerror(char *);  
%}  
  
%token NUM  
  
%left '+' '-'  
  
%left '*' '/'  
  
%right NEGAR  
  
%% // Seção de Regras de Tradução  
  
lines : expr '\n'      { printf("%lf\n", $1); return 0; }  
  
;
```

```
expr : expr '+' expr  { $$ = $1 + $3; }  
    | expr '-' expr  { $$ = $1 - $3; }  
    | expr '*' expr  { $$ = $1 * $3; }  
    | expr '/' expr  { $$ = $1 / $3; }  
    | '(' expr ')'    { $$ = $2; }  
    | '-' expr %prec NEGAR { $$ = -$2; }  
    | NUM  
    ;
```

%% // Seção de Código

```
int yylex() {  
    char C = getchar();  
    if (C == '.' || isdigit(C)) {  
        ungetc(C, stdin);  
        fscanf(stdin, "%lf%c", &yylval, &C);  
        ungetc(C, stdin);  
        return NUM;    }  
    return C;    }
```

// yyerror() e main() não mudam

Exemplo de Programa Yacc

- Outra versão (+ completa):

// Seção de Declarações

```
%{  
#include <ctype.h>  
#include <stdio.h>  
  
#define YYSTYPE double // Define tipo da pilha  
  
int yylex();  
void yyerror(char *);  
%}  
  
%token NUM  
  
%left '+' '-'  
%left '*' '/'  
%right NEGAR  
  
%% // Seção de Regras de Tradução
```

**Definição da
associatividade e da
precedência**

```
lines : expr '\n'      { printf("%lf\n", $1); return 0; }  
      ;
```

```
expr : expr '+' expr  { $$ = $1 + $3; }  
      | expr '-' expr  { $$ = $1 - $3; }  
      | expr '*' expr  { $$ = $1 * $3; }  
      | expr '/' expr  { $$ = $1 / $3; }  
      | '(' expr ')'    { $$ = $2; }  
      | '-' expr %prec NEGAR { $$ = -$2; }  
      | NUM  
      ;
```

Tag usada para
**forçar outra
precedência**

%% // Seção de Código

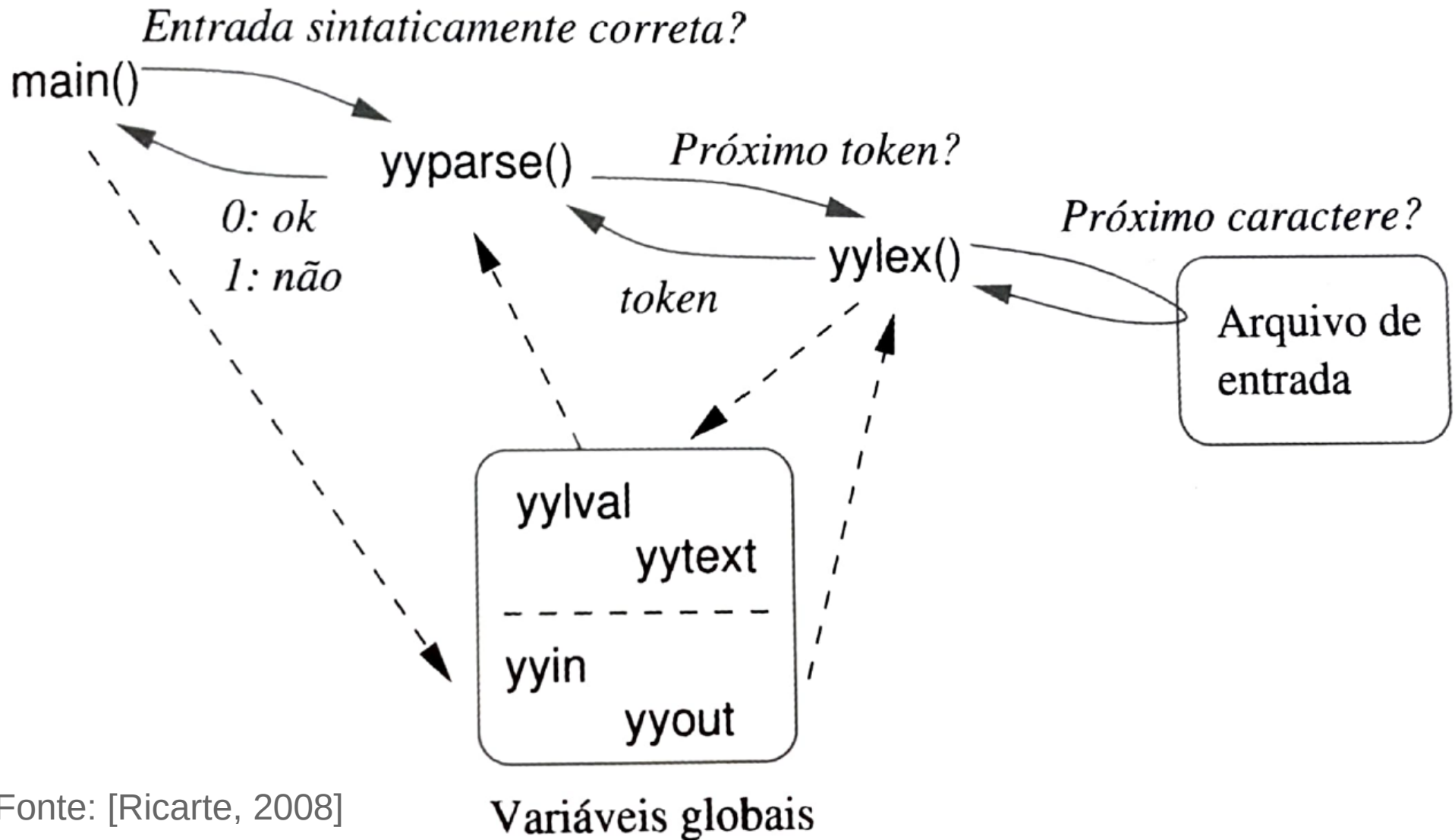
```
int yylex() {  
    char C = getchar();  
    if (C == '.' || isdigit(C)) {  
        ungetc(C, stdin);  
        fscanf(stdin, "%lf%c", &yylval, &C);  
        ungetc(C, stdin);  
        return NUM;    }  
    return C;    }
```

// yyerror() e main() não mudam

Integração Análises Léxica e Sintática

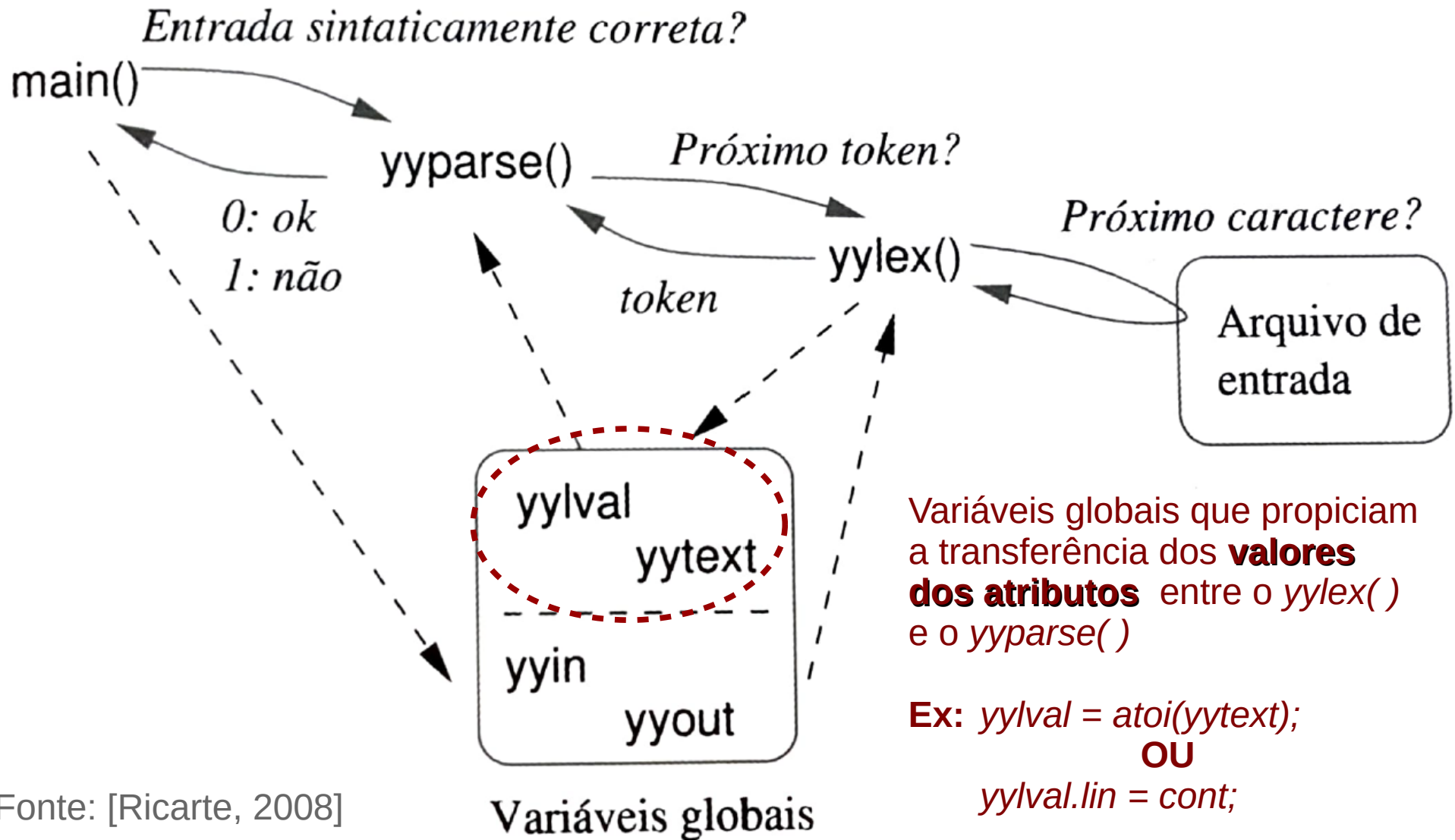
- Gerar um analisador léxico com o nome **yylex()**
 - Pode ser implementado manualmente ou através da ferramenta **lex**
 - Usar o **arquivo cabeçalho com as definições dos *tokens* e as variáveis globais (ex: *yylval* e *yyin*)** para a integração dos módulos
- Gerar um analisador sintático com o nome **yyparse()**
 - Especificar a gramática da linguagem no arquivo com extensão **.y**
 - A integração entre **yylex()** e **yyparse()** exige uma interface em comum:
 - Mesmos **identificadores para os *tokens*** (definidos em um **arq. cabeçalho**)
 - **Variável global *yylval*** deve receber o **valor dos atributos do *token*** retornado
- Compilar e ligar os programas-fonte dos analisadores
 - Analisador léxico não pode ter função **main()** e precisa da **biblioteca -ll**
 - **Ex:** `gcc -o analisador y.tab.c lex.yy.c -ll -ly`

Integração Análises Léxica e Sintática



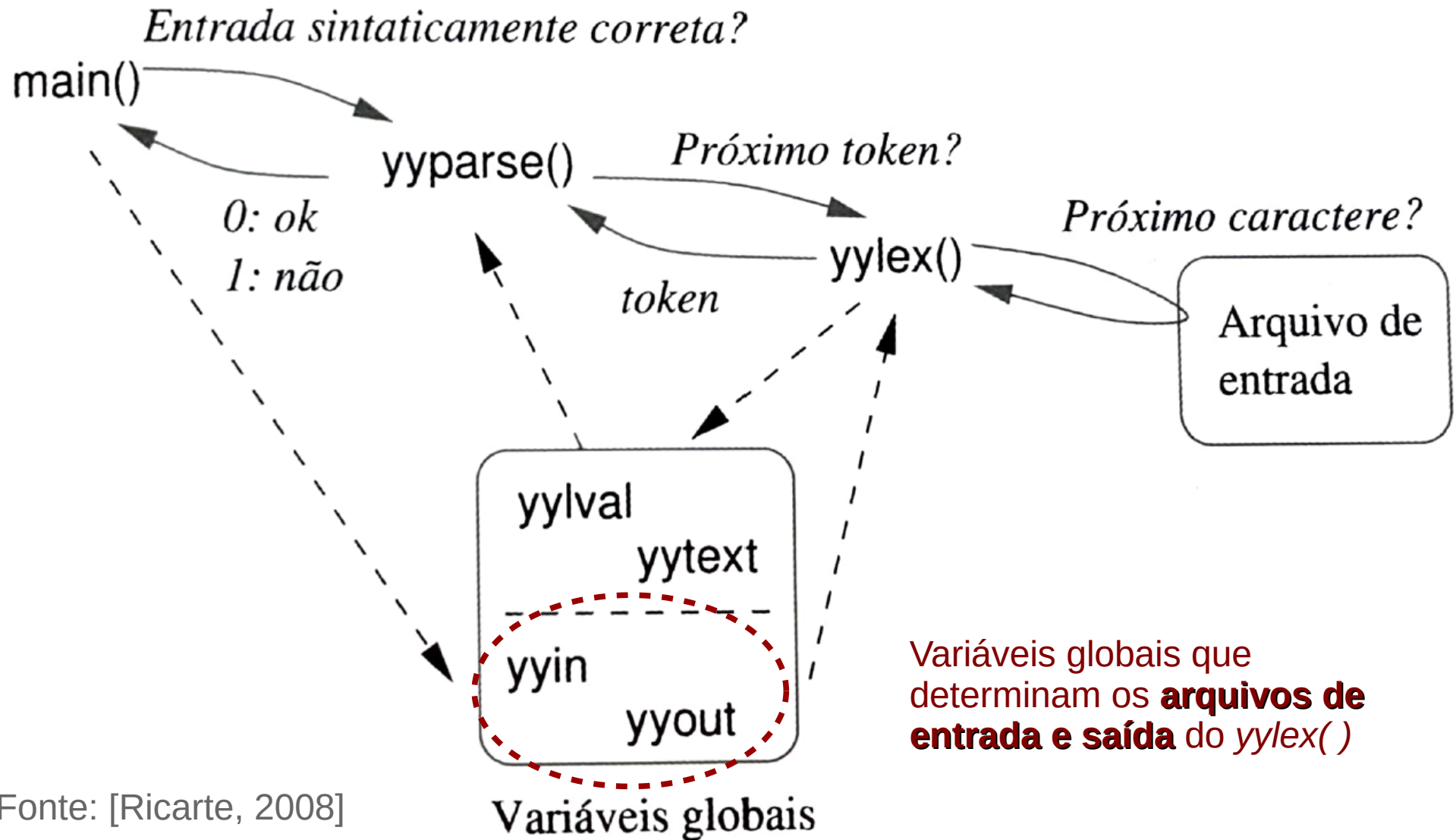
Fonte: [Ricarte, 2008]

Integração Análises Léxica e Sintática



Fonte: [Ricarte, 2008]

Integração Análises Léxica e Sintática



Exemplo de Programa Yacc

• Código Yacc:

```
%{  
    #include "lexer.h" // Analisador lexico  
    #define YYSTYPE double // Define tipo da pilha  
    void yyerror(char *); // Prototipo da funcao de erro  
%}  
  
%token NUM  
%left '+' '-'  
%left '*' '/'  
%right NEGAR  
  
%%  
lines : lines expr '\n' { printf("%.2lf\n", $2); }  
      | lines '\n'      { return 0; }  
      |  
      | error '\n' { yyerror("Erro na ultima linha"); yyerrok; }  
      ;
```

```
expr : expr '+' expr { $$ = $1 + $3; }  
     | expr '-' expr { $$ = $1 - $3; }  
     | expr '*' expr { $$ = $1 * $3; }  
     | expr '/' expr { $$ = $1 / $3; }  
     | '(' expr ')'   { $$ = $2; }  
     | '-' expr %prec NEGAR { $$ = -$2; }  
     | NUM  
     ;  
  
%%  
/* yacc error handler */  
void yyerror(char * s) {  
    fprintf (stderr, "%s\n", s);  
}  
  
int main(void) {  
    return yyparse();  
}
```

Exemplo de Programa Yacc

• Código Yacc:

```
%{  
    #include "lexer.h" // Analisador lexico  
    #define YYSTYPE double // Define tipo da pilha  
    void yyerror(char *); // Prototipo da funcao de erro  
%}
```

```
%token NUM  
%left '+' '-'  
%left '*' '/'  
%right NEGAR
```

%%

```
lines: lines expr '\n' { printf("%.2lf\n", $2); }  
    | lines '\n'      { return 0; }  
    |  
    | error '\n' { yyerror("Erro na ultima linha"); yyerrok; } }  
;
```

Rotina especial que
retorna o analisador
ao seu **modo de
operação normal**

```
expr: expr '+' expr { $$ = $1 + $3; }  
    | expr '-' expr { $$ = $1 - $3; }  
    | expr '*' expr { $$ = $1 * $3; }  
    | expr '/' expr { $$ = $1 / $3; }  
    | '(' expr ')'   { $$ = $2; }  
    | '-' expr %prec NEGAR { $$ = -$2; }  
    | NUM  
    ;
```

%%

```
/* yacc error handler */  
void yyerror(char * s) {  
    fprintf(stderr, "%s\n", s);  
}  
int main(void) {  
    return yyparse();  
}
```

Exemplo de Programa Yacc

• Código Yacc:

```
%{  
    #include "lexer.h" // Analisador lexico  
    #define YYSTYPE double // Define tipo da pilha  
    void yyerror(char *); // Prototipo da funcao de erro  
}%  
  
%token NUM  
%left '+' '-'  
%left '*' '/'  
%right NEGAR  
  
%%  
lines: lines expr '\n' { printf("%.2lf\n", $2); }  
      | lines '\n'      { return 0; }  
      |  
      | error '\n' { yyerror("Erro na ultima linha"); yyerrok; }  
      ;
```

Aplicação principal
invocando o
analisador sintático

```
expr : expr '+' expr { $$ = $1 + $3; }  
      | expr '-' expr { $$ = $1 - $3; }  
      | expr '*' expr { $$ = $1 * $3; }  
      | expr '/' expr { $$ = $1 / $3; }  
      | '(' expr ')'   { $$ = $2; }  
      | '-' expr %prec NEGAR { $$ = -$2; }  
      | NUM  
      ;  
  
%%  
/* yacc error handler */  
void yyerror(char * s) {  
    fprintf (stderr, "%s\n", s);  
}  
  
int main(void) {  
    return yyparse();  
}
```

Exemplo de Programa Yacc

- **Código Lex:**

```
%option noyywrap
```

```
%option outfile="Lexer.c" header-file="Lexer.h"
```

```
% {
```

```
    #define YYSTYPE double // Define tipo yyval (extern YYSTYPE yyval no y.tab.h)
```

```
    #include "y.tab.h" // Biblioteca Analisador Sintatico
```

```
%}
```

```
DIGITO [0-9]+
```

**Integração com o
analisador sintático**

```
%%
```

```
    [t]                { } /* ignora tabulacao e espaco */
```

```
{DIGITO}*[.]?{DIGITO}+ { yyval = atof (yytext); return NUM; }
```

```
    .                  { return yytext[0]; }
```

```
    \n                 { return yytext[0]; }
```

```
%%
```

```
// Seção de Código VAZIA
```

Exemplo de Programa Yacc

- **Execução:**
lex sample_lex_program.l
yacc -d sample_yacc_program.y
gcc lex.yy.c y.tab.c -ll
./a.out

```
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ lex evaluate_exp.l
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ yacc -d evaluate_exp.y
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ cc lex.yy.c y.tab.c -ll
evaluate_exp.l:4:8: warning: type defaults to 'int' in declaration of 'yylval'
[-Wimplicit-int]
extern yyval;
      ^~~~~~
y.tab.c: In function 'yyparse':
y.tab.c:1124:16: warning: implicit declaration of function 'yylex' [-Wimplicit-
function-declaration]
    yychar = yylex ();
              ^~~~~~
y.tab.c:1314:7: warning: implicit declaration of function 'yyerror'; did you me
an 'yyerrok'? [-Wimplicit-function-declaration]
    yyerror (YY_("syntax error"));
    ^~~~~~
    yyerrok
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ ./a.out
Enter the expression
7*(5-3)/2
Result = 7
aashutosh@ubuntu:~/Desktop/lex-Yacc_prgms$ ./a.out
Enter the expression
6/((3-2)*(-5+2))
Result = -2
```

Referências Bibliográficas

- Aho, A.V.; Lam, M.S.; Sethi, R.; Ullman, J.D. Compiladores: Princípios, técnicas e ferramentas, 2ª ed., Pearson, 2008
- Alexandre, E.S.M. Livro de Introdução a Compiladores, UFPB, 2014
- Aluisio, S. material da disciplina “Teoria da Computação e Compiladores”, ICMC/USP, 2011
- Dubach, C. material da disciplina “*Compiling Techniques*”, University of Edinburgh, 2018
- Freitas, R. L. notas de aula - Compiladores, PUC Campinas, 2000
- GeeksforGeeks, *Introduction to YACC*,
<https://www.geeksforgeeks.org/introduction-to-yacc/>
- Menezes, P.B. Linguagens Formais e Autômatos, 6ª ed., Bookman, 2011
- Ricarte, I. Introdução à Compilação, Elsevier, 2008